

Integración SINCRÓNICA vs ASINCRÓNICA:

SINCRÓNICA	ASINCRÓNICA
Las respuestas son inmediatas.	Las respuestas no son inmediatas, sino que suceden un tiempo después.
<u>Situación:</u> [A] le pide resolver algo a [B]. Mientras [B] lo resuelve... [A] queda en espera activa, aguardando la respuesta de [B] para seguir operando.	<u>Situación:</u> [A] le pide resolver algo a [B]. Mientras [B] lo resuelve... [A] sigue operando con otra cosa, aguardando la respuesta de [B].

Una cosa es que un proceso sea SINCRÓNICO o ASINCRÓNICO y otra cosa es que una integración sea SINCRÓNICA o ASINCRÓNICA...

Ejemplo: se quiere hacer una factura electrónica y nos integramos con un tercero.

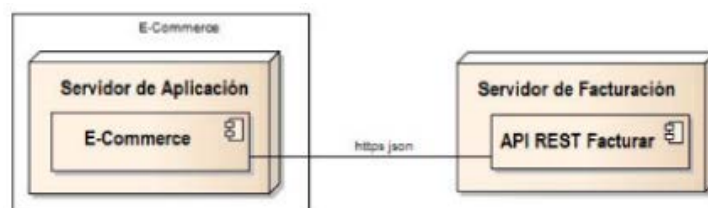
Para eso, enviamos los datos de la factura al servicio de facturación.

Luego:

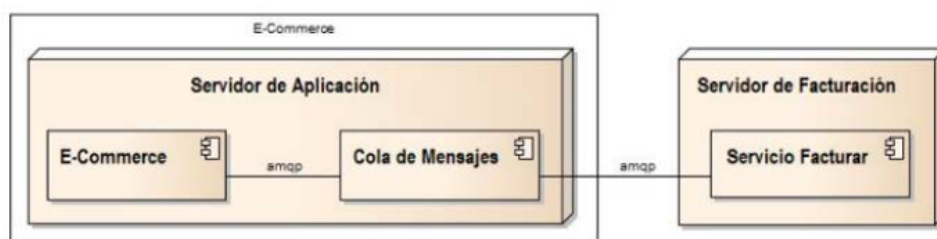
- Si el servicio de facturación genera la factura y nos envía el PDF, entonces...
... el proceso es 100% SINCRÓNICO, tanto del punto de vista del usuario como del desarrollo: nos quedamos esperando el PDF.
- Si el servicio de facturación responde "OK, recibí bien los datos" (una especie de ACK) y, más tarde, cuando el servicio pueda, recién ahí genera la factura y envía el PDF, entonces...
... el proceso referido a la conexión (enviar los datos y recibir un ACK) es SINCRÓNICO; pero si vemos el proceso a nivel macro, a nivel negocio, es ASINCRÓNICO → el único sincronismo que hay es el ACK (un acuse de recibo); no es que recibimos el PDF inmediatamente.

API REST:

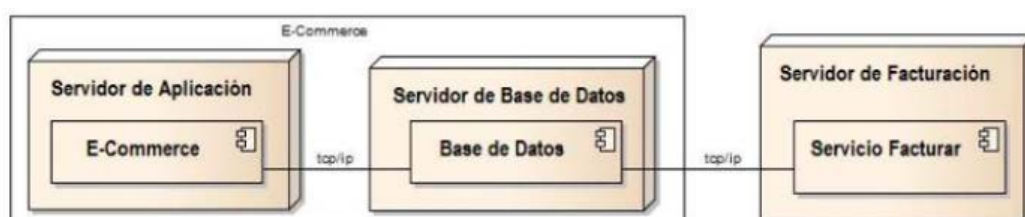
- Nuestro componente hace una solicitud a una **API REST** externa para que ésta resuelva las operaciones requeridas.
- Si la **API REST** responde con lo solicitado inmediatamente → SINCRÓNICA.
Si la **API REST** responde con un ACK (indicando que la solicitud fue recibida con éxito) y recién más tarde, cuando pueda, responder con lo solicitado → ASINCRÓNICA.
- *Esta estrategia debe ser evaluada únicamente si el sistema es externo.*

**Cola de Mensajes:**

- La **cola de mensajes** va recibiendo solicitudes de operaciones y, en la medida que pueda, irá respondiendo con lo solicitado → ASINCRÓNICA.
- *Esta estrategia debe ser evaluada únicamente si el proceso es ASINCRÓNICO.*

**Base de Datos Compartida:**

- Nuestro componente va insertando registros en nuestra **DB compartida** con un componente externo, quien realizará las operaciones requeridas.
- Estos registros son, de alguna manera, indicadores de solicitudes de operaciones.
- Eventualmente, el componente externo se conectará a la **DB compartida**, revisará si hay nuevos registros y, si los hubiera, realizará las operaciones → ASINCRÓNICA.
- *Esta estrategia se la puede pensar para un componente interno de nuestra organización (como un esquema centrado en datos), pero no para integrarse con un sistema externo.*
No obstante, si así se decide, los componentes externos deben tener permisos para escribir en la DB y también deben conocer las estructuras de las tablas que escriben.



Ventajas y Desventajas de cada una:

	API REST	Cola de Mensajes	DB Compartida
MANTENIBILIDAD	▼ En caso de cambiar algo en la API REST, habrá que cambiar algo en nuestro componente que se conecta a ella.	▼ Es un componente más para mantener en nuestro sistema.	▼▼▼ Es un componente más para mantener en nuestro sistema. Hay problemas con la integridad de los datos. Es complejo si hay que actualizar la estructura de una tabla de la DB.
INTEROPERABILIDAD	▼ Si se quieren agregar otros componentes externos en un futuro, hay que volver a desarrollar algo específico para cada integración.	▲▲▲ Si se quieren agregar otros componentes externos en un futuro, la misma cola de mensajes nos sirve para integrarnos a estos nuevos servicios.	▼ Si se quieren agregar otros componentes externos en un futuro, hay que volver a desarrollar algo específico para cada integración.
DISPONIBILIDAD	▼ Si el componente externo cerrara durante cierto tiempo (por tareas de mantenimiento, por ejemplo), nuestro sistema dejará de funcionar.	▲▲▲ Si el componente externo cerrara durante cierto tiempo (por tareas de mantenimiento, por ejemplo), nuestro sistema seguirá funcionando normalmente.	▼ Puede ser un punto único de falla.
PERFORMANCE	-	▲	▼ Puede ser un cuello de botella.
CONFIABILIDAD	-	▲ Es muy confiable en el envío de mensajes.	-
COSTOS	-	▲ Implementación costosa.	-
ACOPLAMIENTO	-	▼ Menor acoplamiento entre los sistemas que se integran.	▲ Es la alternativa que queda más acoplada a nuestro sistema.

API vs Servicio Web

ARQUITECTURA

API	Servicio Web
Se comunican de software a software.	Se comunican de máquina a máquina.
Trabaja con JSON, permitiendo a consumidores/proveedores de servicios serializar y deserializar (empaquetar y desempaquetar datos) mucho más rápido.	Trabaja con XML, sin las ventajas de JSON.

REST vs SOAP

ARQUITECTURA

REST	SOAP
Se comunica vía protocolo HTTP, aprovechando las ventajas de los métodos GET, POST, PUT, DELETE, ...	Se comunica vía protocolo HTTP, pero también FTP, POP3, TCP y colas de mensajería.
Puede transmitir XML, JSON, archivos binarios, texto, ...	Solamente transmite XML.